CAPSTONE GRADUATION PROJECT

Machine Learning

Capstone on E-Commerce

Classification · Regression · Clustering — Pick Your Model, Own Your Problem

5 Models to Choose From	**Each Student Picks ONE**
Jupyter Notebook	**LinkedIn Delivery**

TechTrek, 28 Ibrahim Sherif St ,Mostafa Kamel Alexandria Phone +201157714484, techtrek.eg@gmail.com

Overview

How This Capstone Works

Every student goes through the same data preparation and EDA. Then each student picks ONE model to build, tune, and present as their own capstone deliverable.

Shared Phase (Everyone)

Load data, audit quality, clean, preprocess, engineer features, do EDA. Same notebook base for all.

Individual Phase (Your Model)

Pick one of 5 models. Train, tune, evaluate, interpret. This is YOUR unique contribution.

Delivery Phase (Everyone)

Finalize notebook, export model and data, record LinkedIn video, publish post.

One Model Per Student

Each student must choose a different model. Coordinate with your team. If there are more students than models, some models can have 2 students — but they must work independently and produce separate notebooks.

Dataset

The Dataset

A comprehensive e-commerce dataset with orders, customers, products, pricing, marketing, delivery, returns, inventory, and supplier information.

30+ Columns	**100K+** Rows
5 Model Options	**3** ML Types
6 Business Areas	

Column Mapping

Map the Kaggle dataset columns to the schema defined in this brief. Some columns (competitor pricing, weather) may need simulation or supplementary data.

Domain

Business Domain — 6 Areas

Business Area	Focus
Customer	Demographics, locations, spending patterns, satisfaction
Sales	Revenue, quantity, discount impact, product performance
Marketing	Campaign attribution, source effectiveness
Logistics	Courier performance, delays, delivery status
Returns	Return reasons, complaints, refunds, ratings
Inventory	Stock levels, lead times, price competitiveness

Choose

Pick Your Model

Each student claims one model. Read the task cards on the next slides carefully before choosing.

#	Model	Type
1	Return Prediction	Classification
2	Delay Prediction	Classification
3	Rating Prediction	Regression
4	Customer Segments	Clustering
5	Revenue Prediction	Regression

How to Choose

Pick based on what interests you most: customer behavior (Model 1 or 4), operations (Model 2), satisfaction (Model 3), or business forecasting (Model 5). All are equal in difficulty and grading weight.

Model 1 Task

Return Prediction

Binary Classification

Predict whether an order will be returned (Yes/No) at the time the order is placed. This helps the business identify high-risk orders early.

Your Specific Tasks

- Define target: Returned = Yes (1) / No (0)
- Select features: customer, product, pricing, channel, stock — NO return reason, complaint, or refund fields
- Train and compare: Logistic Regression, Decision Tree, Random Forest, KNN
- Handle class imbalance: compare class_weight vs no adjustment
- Evaluate: Accuracy, Precision, Recall, F1, ROC-AUC
- Show: confusion matrix, feature importance from best tree model
- Write: 3+ business insights from feature importance and error patterns

Example Insight

> *"Orders with discount >15% have 28% return rate vs 9% for no-discount orders. The model learned this — discount is a top-3 feature. Recommendation: investigate product quality in high-discount categories."*

Model 2 Task

Delivery Delay Prediction

Binary Classification

Predict whether an order will be delivered late (actual > expected days). This helps set realistic delivery expectations and allocate logistics resources.

Your Specific Tasks

- Create target: is_late = 1 if actual_delivery > expected_delivery, else 0
- Select features: courier, expected days, weather, season, area, product, month/hour — NO actual_delivery_days or delivery_status
- Train and compare: Logistic Regression, Decision Tree, Random Forest, KNN
- Handle class imbalance: compare class_weight vs no adjustment
- Evaluate: Accuracy, Precision, Recall, F1, ROC-AUC
- Show: confusion matrix, feature importance, delay rate by courier
- Write: 3+ business insights — which courier/area/season is riskiest

Example Insight

> *"Courier B has 42% late rate vs 18% for Courier A. After controlling for area, Courier B is still 2.3x more likely to be late. Recommendation: renegotiate SLA with Courier B or reduce their order volume."*

Model 3 Task

Customer Rating Prediction

Regression

Predict the customer rating (1-5) an order will receive. This helps identify orders at risk of low ratings before the customer reviews.

Your Specific Tasks

- Define target: Rating (1-5). Drop rows where rating is missing.
- Select features: delivery delay, delivery status, product category, price gap, discount, courier, channel, return status
- Train and compare: Linear Regression, Decision Tree Regressor, Random Forest Regressor, KNN Regressor
- Check target distribution: is it skewed? Compare raw vs log-transformed target
- Evaluate: MAE, RMSE, R-squared
- Show: predicted vs actual scatter plot, residual plot, error by product category
- Write: 3+ business insights — what drives low ratings

Example Insight

> *"Each additional day of delay reduces predicted rating by 0.3 points on average. Orders delayed by 3+ days average 2.1 stars vs 4.2 for on-time orders. Recommendation: prioritize reducing delays for high-value orders."*

Model 4 Task

Customer Segmentation

Unsupervised Clustering

Discover natural customer segments based on spending behavior, order patterns, and demographics. No target variable — let the data reveal groups.

Your Specific Tasks

- Build RFM features per customer: Recency, Frequency, Monetary value, avg order value
- Add: age group, preferred channel, preferred category, return rate, area
- Preprocess: StandardScaler, log-transform monetary features to handle skewness
- Train and compare: K-Means (K=2 to 8), DBSCAN, Hierarchical Clustering
- Find best K: elbow method (inertia) + silhouette score
- Evaluate: Silhouette Score, Davies-Bouldin Index, segment size balance
- Profile each segment: name it, describe it, calculate key stats per segment
- Show: elbow plot, silhouette plot, segment profiles table, segment size chart
- Write: 3+ business insights — who are the best/worst customers

Example Insight

> *"Segment 'High-Value Loyalists' (12% of customers) generates 41% of revenue with 4.6 avg rating and 6% return rate. Segment 'Bargain Hunters' (28%) generates 15% of revenue with 3.1 avg rating and 31% return rate. Recommendation: different retention strategies per segment."*

Model 5 Task

Revenue Prediction

Regression

Predict the net revenue of an order. This helps with revenue forecasting, pricing decisions, and promotion planning.

Your Specific Tasks

- Define target: net_revenue = unit_price × quantity × (1 - discount_percent/100)
- Select features: product category, brand, unit price, quantity, discount, channel, season, payment method, customer age group
- Check target distribution: likely right-skewed — compare raw vs log-transformed
- Train and compare: Linear Regression, Decision Tree Regressor, Random Forest Regressor, KNN Regressor
- Evaluate: MAE, RMSE, R-squared
- Show: predicted vs actual scatter, residual plot, error by price range, error by category
- Write: 3+ business insights — what drives high/low revenue orders

Example Insight

> *"The model's largest errors are on bulk orders (quantity >5), where it under-predicts revenue by 22% on average. Electronics category has highest avg revenue but also highest variance. Recommendation: build separate pricing rules for bulk orders."*

Shared Phase

What EVERY Student Does

Before you split into individual models, every student completes this shared foundation.

Load & Preserve

- Read CSV, preserve raw copy
- Show shape, head, tail
- Set random seed

Audit

- info(), describe() for all columns
- Missing count and percentage
- Exact duplicates, duplicate Order_ID
- Hidden nulls: 'NA', 'N/A', 'Null', '-', blanks

Preprocess

- Clean column names (snake_case)
- Fix data types: dates, numerics, categoricals
- Standardize text labels (Online/online → Online)
- Handle missing values (document each decision)
- Remove exact duplicates after inspection
- Validate business rules (age, prices, dates)

Feature Engineering

- net_revenue, gross_revenue, discount_amount
- delivery_delay, is_late
- price_gap, is_pricier_than_competitor
- age_group, order_month, order_weekday, order_hour
- is_weekend, low_stock_flag

Important: Null Handling Rules

> Return fields null when Returned=No → keep null (not an error). Rating null = no review → keep null. Campaign null = no campaign → fill 'No Campaign'. Do NOT fill return fields with fake values.

Pipeline

The ML Pipeline

Step	Phase	Purpose
1	Load	Read, raw copy
2	Audit	Quality check
3	Clean	Types, text, nulls
4	Features	Engineer new cols
5	EDA	Visualize data
6	Split	Train / test
7	Encode	Scale, encode
8	Train	Your model
9	Tune	Grid search
10	Evaluate	Metrics, plots

Steps 1-7 Are Shared, Steps 8-10 Are YOUR Model

Everyone does the same data preparation. From step 8 onward, you work on your chosen model independently. Your notebook must include all 10 steps.

Schema

Dataset Schema — Part 1

Group	Columns	Used By Model
Order & Time	Order_ID, Order_Date, Order_Time	All — time features, split key
Customer	Customer_ID, Gender, Age, Area	1, 4, 5 — demographics, RFM
Store & Channel	Branch, Sales_Channel	1, 2, 3, 5 — channel features
Product	Product_Domain, Category, Product_ID, Name, Brand	1, 3, 5 — product features
Sales & Pricing	Unit_Price, Quantity, Discount_Percent, Competitor_Price	1, 3, 5 — pricing features, revenue target
Payment & Marketing	Payment_Method, Marketing_Source, Campaign_Name	1, 5 — payment/marketing features

Schema Continued

Dataset Schema — Part 2

Group	Columns	Used By Model
Delivery	Courier, Expected_Delivery_Days, Actual_Delivery_Days, Delivery_Status	2 (target), 3 (feature)
Returns & Feedback	Returned, Return_Reason, Complaint_Text, Request_Date, Refund_Method, Rating	1 (target=Returned), 3 (target=Rating)
Inventory & Supplier	Stock_Available_Before_Sale, Supplier, Lead_Time_Days	1, 2 — stock/supplier features
External	Weather, Season	2 — weather/season features

Leakage — Read Carefully

> Model 1: Cannot use Return_Reason, Complaint_Text, Request_Date, Refund_Method — they only exist AFTER a return happens.
>
> Model 2: Cannot use Actual_Delivery_Days or Delivery_Status — they ARE the target information.
>
> Model 3: Be careful with Returned status — check if rating happens before or after return decision in the data.
>
> Model 4: No target variable, but be careful not to include future information in RFM calculations.

Exploration

Data Exploration — Everyone

Standard Checks

- Shape, info(), describe()
- Missing count/percentage per column
- Exact duplicates, duplicate Order_ID
- Hidden nulls: 'NA', 'N/A', 'Null', '-', blanks
- Unique values for all categoricals
- Min/max/quartiles for all numerics

Visual Exploration

- Target distribution for YOUR model's target
- Class imbalance check (if classification)
- Top 10 features vs target (bar charts / box plots)
- Correlation heatmap for numeric features
- Orders over time (line chart)
- Revenue by category, channel, area

```
# Required exploration code
print(df.shape)
display(df.head())
display(df.tail())
df.info()
display(df.describe(include='all').T)
missing = pd.DataFrame({
'count': df.isna().sum(),
'percent': df.isna().mean().mul(100)
}).sort_values('percent', ascending=False)
print(missing)
print('Exact duplicates:', df.duplicated().sum())
print('Duplicate Order_ID:', df.duplicated('order_id', keep=False).sum())
# Your target distribution — replace with your model's target
print(df['returned'].value_counts(normalize=True))
```

Preprocessing

Data Cleaning & Encoding

Clean

- snake_case columns
- Strip whitespace, unify casing
- Merge: Online/online → Online
- Standardize Yes/No, gender, branch
- Document every mapping

Fix Types

- Dates → datetime
- Numerics → numeric (errors='coerce')
- Categoricals → 'category' dtype
- Validate: age range, prices > 0, qty > 0
- Discount scale: detect 0-100 vs 0-1

Missing Values

- Return fields (Returned=No) → keep null
- Rating null → keep null (drop for Model 3 only)
- Campaign null → 'No Campaign'
- Audit table: column, count, action, reason

Encode for ML

- Binary: Yes/No → 1/0
- Low-card: gender, channel → OneHotEncoder
- High-card: category, brand, area → LabelEncoder (or drop if >50 unique)
- Fit encoder on train, transform test
- Scale numeric features with StandardScaler

Features

Feature Engineering — Everyone

```
# Revenue
df['gross_revenue'] = df['unit_price'] * df['quantity']
discount_rate = df['discount_percent'].copy()
if discount_rate.max(skipna=True) > 1:
discount_rate = discount_rate / 100
df['discount_amount'] = df['gross_revenue'] * discount_rate
df['net_revenue'] = df['gross_revenue'] - df['discount_amount']
# Delivery
df['delivery_delay'] = df['actual_delivery_days'] - df['expected_delivery_days']
df['is_late'] = (df['delivery_delay'] > 0).astype(int)
# Pricing
df['price_gap'] = df['unit_price'] - df['competitor_price']
df['is_pricier'] = (df['price_gap'] > 0).astype(int)
# Time
df['order_month'] = df['order_date'].dt.month
df['order_weekday'] = df['order_date'].dt.dayofweek
df['order_hour'] = df['order_time'].dt.hour
df['is_weekend'] = df['order_weekday'].isin([5,6]).astype(int)
# Customer
df['age_group'] = pd.cut(df['age'], bins=[0,18,25,35,45,55,100],
labels=['U18','18-25','26-35','36-45','46-55','55+'])
# Stock
df['low_stock_flag'] = (df['stock_before_sale']
```

For Model 4 (Clustering) Only

You also need to compute RFM features per customer: recency (days since last order), frequency (order count), monetary (total spend), avg order value, return rate. Do this AFTER the train/test split — compute on training customers only.

Split & Encode

Train/Test Split & Preparation

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder, OneHotEncoder
# 1. Define YOUR target (example for Model 1)
y = df['returned'].map({'Yes': 1, 'No': 0})
X = df.drop(columns=['returned', 'return_reason', 'complaint_text',
'return_request_date', 'refund_method', 'order_id',
'customer_id', 'product_id', 'order_date', 'order_time'])
# 2. Split
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y
)
# 3. Encode categoricals
cat_cols = X_train.select_dtypes(include='object').columns
num_cols = X_train.select_dtypes(include='number').columns
# OneHotEncode low-cardinality, LabelEncode high-cardinality
for col in cat_cols:
if X_train[col].nunique()
```

Algorithms

Algorithms by Model Type

Model Type	Algorithms to Try	Why These
Classification (Models 1, 2)	LogisticRegression DecisionTreeClassifier RandomForestClassifier KNeighborsClassifier	Covers linear, tree, distance-based. Easy to interpret and compare.
Regression (Models 3, 5)	LinearRegression DecisionTreeRegressor RandomForestRegressor KNeighborsRegressor	Same logic as classification. Linear baseline + tree + distance.
Clustering (Model 4)	KMeans (K=2 to 8) DBSCAN AgglomerativeClustering	K-Means for well-separated clusters, DBSCAN for irregular shapes, Hierarchical for dendrogram view.

What About XGBoost / SVM / Neural Networks?

Not required. These 4 algorithms per type are sufficient for a capstone at this level. If you want to add one more as a bonus, you can — but the 4 listed are the core requirement. Quality of analysis beats quantity of algorithms.

Tuning

Hyperparameter Tuning

For your best-performing algorithm only. Don't tune all 4 — tune the winner.

```
from sklearn.model_selection import GridSearchCV
# Example: tuning RandomForest (Classification)
param_grid = {
'n_estimators': [100, 200],
'max_depth': [5, 10, None],
'min_samples_split': [2, 5],
'min_samples_leaf': [1, 2]
}
grid = GridSearchCV(
RandomForestClassifier(random_state=42),
param_grid, cv=5, scoring='f1_macro', n_jobs=-1
)
grid.fit(X_train, y_train)
print(f'Best params: {grid.best_params_}')
print(f'Best F1: {grid.best_score_:.4f}')
best_model = grid.best_estimator_
```

Tuning Rules

> Use stratified CV for classification. Use F1-macro as scoring for imbalanced classification (not accuracy). For regression, use neg_root_mean_squared_error. Keep the grid small — 2-3 values per param max.

Evaluation

Evaluation Metrics by Model Type

Model	Type	Required Metrics	Required Plots
1. Return	Classification	Accuracy, Precision, Recall, F1 (macro), ROC-AUC	Confusion matrix, ROC curve, feature importance
2. Delay	Classification	Accuracy, Precision, Recall, F1 (macro), ROC-AUC	Confusion matrix, ROC curve, feature importance
3. Rating	Regression	MAE, RMSE, R-squared	Predicted vs actual scatter, residual plot
4. Segments	Clustering	Silhouette Score, Davies-Bouldin Index	Elbow plot, silhouette plot, segment profile table
5. Revenue	Regression	MAE, RMSE, R-squared	Predicted vs actual scatter, residual plot

Rules

> For imbalanced classification: do NOT report accuracy as your main metric — lead with F1-macro and ROC-AUC. For regression: always show the residual plot — it reveals problems that R^2 hides. For clustering: always show the elbow plot — it justifies your K choice.

Comparison

Model Comparison Table

Compare all 4 algorithms you tried for YOUR model. Show which one won and why.

Algorithm	Metric 1	Metric 2	Metric 3	Tuned?	Selected?
LogisticRegression	TBD	TBD	TBD	No	—
DecisionTree	TBD	TBD	TBD	No	—
RandomForest	TBD	TBD	TBD	Yes	✓
KNN	TBD	TBD	TBD	No	—

```
# Example comparison code
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
results = []
for name, model in models.items():
model.fit(X_train, y_train)
preds = model.predict(X_test)
results.append({
'Model': name,
'Accuracy': accuracy_score(y_test, preds),
'F1_macro': f1_score(y_test, preds, average='macro'),
'ROC-AUC': roc_auc_score(y_test, model.predict_proba(X_test)[:,1])
})
pd.DataFrame(results).sort_values('F1_macro', ascending=False)
```

Visualization

Visualization Standards

EDA Visuals

- Target distribution (bar chart)
- Feature vs target (bar/box plots)
- Correlation heatmap (numeric features)
- Revenue/orders over time (line chart)
- Category counts (horizontal bar)

ML Visuals

- Classification: Confusion matrix (heatmap), ROC curve, feature importance bar
- Regression: Predicted vs actual (scatter), residuals (scatter), error by category
- Clustering: Elbow plot, silhouette plot, segment profiles, segment sizes

Forbidden

> 3D charts. Pie charts with many slices. Accuracy as main metric for imbalanced data. Charts without titles. Charts without axis labels or units. Charts without a following Markdown insight. Word clouds as analysis.

Insight Template

Mandatory Insight Structure

Every visual must be followed by this exact structure. No shortcuts, no exceptions.

```
### Insight
- **What happened?** State the finding using numbers.
- **ML meaning:** How does this affect the model or feature selection?
- **Business meaning:** Why does this matter to the business?
- **Action / recommendation:** What should the business do?
- **Limitation:** Missing data, small sample, correlation vs causation.
```

Example (Classification)

> *What happened? Discount >15% is the 3rd most important feature. Orders with high discount have 28% return rate vs 9% for no-discount.*
> *ML meaning: The model correctly learned this pattern — removing discount drops F1 from 0.72 to 0.65.*
> *Business meaning: High discounts may attract lower-quality purchases.*
> *Action: Investigate product quality in high-discount categories. Add post-purchase follow-up for high-discount orders.*
> *Limitation: Category confound — high-discount categories may inherently have higher return rates.*

Notebook Structure

How Your Notebook Must Be Organized

#	Section	Content
1	Title & Info	Your name, chosen model, date, dataset link
2	Business Understanding	Domain, your model's problem, why it matters, success criteria
3	Imports	All libraries, random seed, display settings
4	Load Data	Read file, preserve raw copy, shape, sample
5	Data Audit	Info, describe, missing, duplicates, hidden nulls
6	Preprocessing	Clean, fix types, standardize, handle missing, validate rules
7	Feature Engineering	All derived features with code and explanation
8	EDA	Target distribution, feature-target relationships, correlations
9	Train/Test Split	Split, encode, scale — with code
10	Model Training	Train 4 algorithms, compare results
11	Tuning	Grid search on best algorithm, report best params
12	Final Evaluation	Metrics, required plots, feature importance
13	Error Analysis	Where does the model fail? Any patterns?
14	Key Findings	5-8 findings with the insight template
15	Recommendations	Business actions based on findings
16	Limitations	What's missing, what could be improved
17	Export	Clean data, model file, visuals

Files

Folder Structure & Deliverables

```
capstone_project/
│
├── data/
│
├── raw_orders.csv
│
└── clean_orders.csv
│
├── notebook/
Capstone_Model[X]_Analysis.ipynb
│
├── model/
│
└── best_model.pkl
│
├── visuals/
│
├── 01_confusion_matrix.png
│
├── 02_roc_curve.png
│
└── ...
│
├── reports/
│
└── model_comparison.csv
│
└── README.md
```

Capstone_Model[X]_Analysis.ipynb

Full notebook: all 17 sections. Restart & Run All must pass.

Capstone_Clean_Orders.csv

Cleaned dataset with all derived features.

best_model.pkl

Your best tuned model, saved with joblib.

visuals/ folder

All key charts exported as PNG with numbered filenames.

LinkedIn Video

60-120 seconds: your model, 1-2 plots, 1 insight.

LinkedIn Post

Describe your model, metrics, and key finding.

LinkedIn

Video & Post

Video (60-120 sec)

- Introduce yourself: "This is my ML capstone"
- Say which model you chose and why
- Show your notebook: cleaning, features, training
- Show 1-2 key plots and explain the finding
- State your best metric number
- State the #1 business recommendation
- Mention what you learned about ML

Post Template

> *I am excited to share my Machine Learning capstone project.*
>
> *I chose to build a [Model Name] model on an e-commerce dataset to [business problem].*
>
> *I compared 4 algorithms: Logistic Regression, Decision Tree, Random Forest, and KNN. The best model was [Winner] with [F1/AUC/RMSE/R^2] of [X.XX].*
>
> *Key finding: [your strongest insight in one sentence].*
>
> *The project included full data cleaning, feature engineering, model comparison, hyperparameter tuning, and business interpretation.*
>
> *#MachineLearning #Python #ScikitLearn #Classification #Regression #Clustering #Capstone #JupyterNotebook*

Evaluation

Grading Rubric — 100 Points

Weight	Evaluation Area	Success Criteria
10%	Business Understanding	Clear problem statement, why this model matters, success criteria
10%	Data Exploration & Audit	Shape, types, missing, duplicates, target distribution, imbalance check
15%	Preprocessing & Feature Engineering	Clean, encode, scale, derived features, no leakage, documented
10%	EDA	Target distribution, feature-target analysis, correlations, each with visual + insight
25%	Model Training, Comparison & Tuning	4 algorithms trained, comparison table, GridSearch on best, correct metrics
10%	Evaluation & Error Analysis	Required plots, metric interpretation, where model fails, feature importance
8%	Insights & Recommendations	5-8 findings with template, business actions, limitations
7%	Notebook Quality + Deliverables	Organized, reproducible, seeds set, no broken cells, all files submitted
5%	LinkedIn Video + Post	Clear explanation of model, shows plots, states metric, mentions learning

Checklist

Final Submission Checklist

- Chosen model declared in notebook title
- Business problem and success criteria stated
- Dataset link included
- Raw data preserved
- Full audit: shape, info, describe, missing, duplicates
- All text labels standardized
- All types corrected
- Missing values documented and handled
- No leakage: no future features in training
- All derived features created and explained
- Target distribution analyzed
- Feature-target relationships visualized
- Train/test split with stratification (if classification)
- Encoding and scaling done correctly
- 4 algorithms trained and compared
- Comparison table filled with actual metrics
- Best algorithm tuned with GridSearchCV
- All required plots present and labeled
- Feature importance shown (tree models)
- Error analysis: where does model fail
- 5-8 insights with full template
- Business recommendations included
- Limitations section included
- Model exported as .pkl
- Notebook restarts and runs without errors
- Random seed set
- LinkedIn video recorded (60-120 sec)
- LinkedIn post published with metrics

Pick Your Model. Own Your Problem.

This capstone proves you can take data, build a model, and explain what it means for the business. Not 7 models. Not deployment. One model, done right.

4 Algorithms	**1** Tuned Winner
5-8 Insights	**0** Leakage
1 Business Story	

Good luck to all teams.